PROJECT REPORT

AutoJudge: Development of an AI-Based Difficulty Assessment Engine for Competitive Programming Problems

Submitted by:
Chehek Agrawal
23117012

Abstract

In the domain of competitive programming, accurately gauging the difficulty of a problem is crucial for user progression and platform organization. Traditional methods rely on manual tagging by problem setters, which is often prone to bias and latency. This project introduces **AutoJudge**, a machine learning framework designed to infer the difficulty level (Easy/Medium/Hard) and a granular difficulty rating of coding problems using their natural language descriptions.

The proposed system employs a robust NLP pipeline integrated with a specialized feature extraction module. By analyzing text for mathematical density, specific algorithmic terminology, and structural metrics, the model constructs a comprehensive feature vector. Through rigorous comparative testing of multiple algorithms, a **Random Forest Ensemble** emerged as the optimal solution, yielding a classification accuracy of **55.04%** and a Regression Mean Absolute Error (MAE) of **1.68**. The final predictive engine is accessible via a modern Streamlit web interface.

---

1. Project Background

1.1 The Challenge

Educational coding platforms (e.g., Codeforces, LeetCode) host vast repositories of problems. Assigning a difficulty tag to these problems is a bottleneck because it requires:

1. Expert human review.
2. Analysis of user submission rates over time.

This project aims to eliminate this bottleneck by automating the assessment process, allowing for instant difficulty prediction based solely on the problem's text statement.

1.2 Core Objectives

- **Pipeline Development:** To construct a data processing pipeline that converts raw text into machine-readable vectors.
- **Feature Engineering:** To design custom "meta-features" that capture the specific nuances of algorithmic complexity (e.g., math symbols).
- **Model Benchmarking:** To compare linear models against ensemble methods for both classification and regression tasks.
- **Deployment:** To provide a visual, interactive web tool for end-users.

---

2. Data Analysis & Exploration

The study is based on a dataset comprising **4,112 algorithmic problems** stored in JSONL format. The data includes the problem title, description, and input/output constraints.

2.1 Data Distribution

An Exploratory Data Analysis (EDA) highlighted a significant class imbalance, reflecting the real-world nature of competitive programming where complex problems outnumber introductory ones.

- **Total Corpus:** 4,112 samples
- **Hard:** 1,941 samples (Dominant class)
- **Medium:** 1,405 samples
- **Easy:** 766 samples

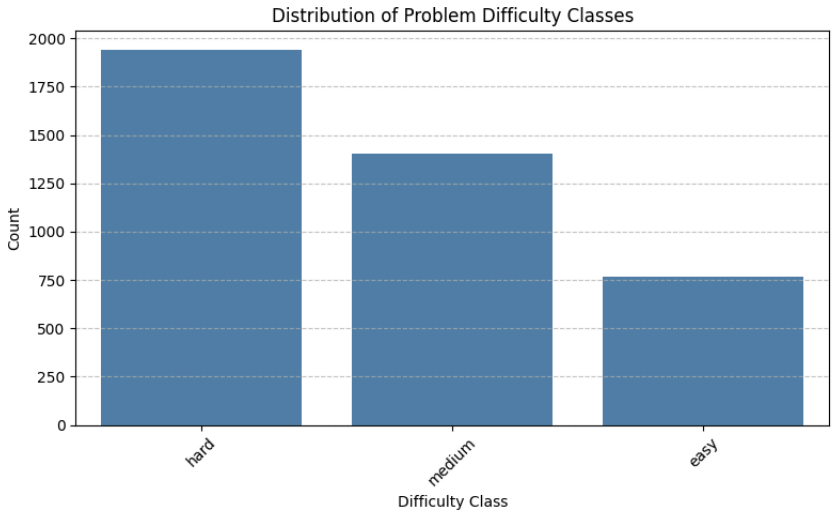

Figure 1: *Bar chart illustrating the imbalance in difficulty categories.*

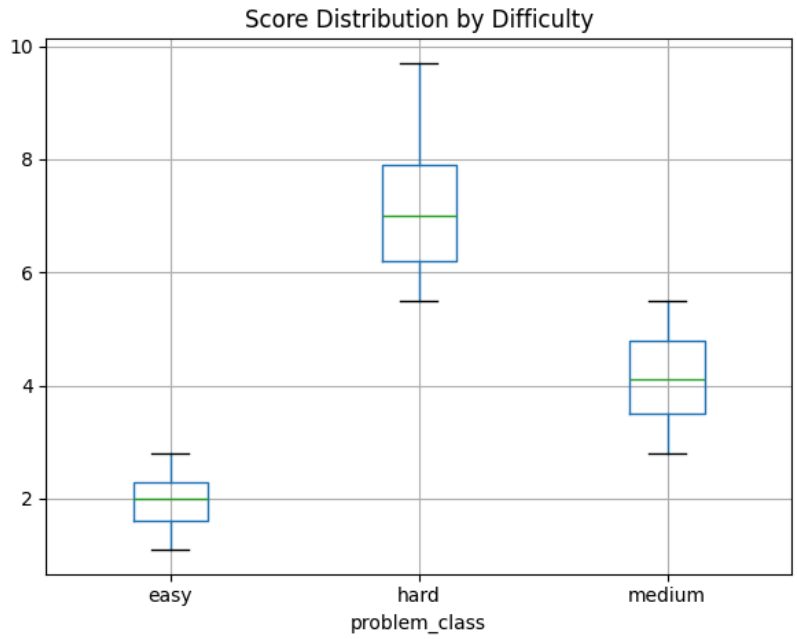

Figure 2: *Statistical distribution of the numerical difficulty scores.*

To ensure the regression models received consistent data, missing difficulty scores were filled using the dataset's median value of **5.2**.

---

3. Implementation Methodology

The project architecture is modular, separating data processing, feature extraction, and model training into distinct phases.

3.1 Text Preprocessing

Raw problem descriptions contain noise. The following cleaning steps were automated:

1. **Concatenation:** Merging Title, Description, and I/O into a single text block.
2. **Normalization:** Converting all text to lowercase to ensure consistency.
3. **Filtration:** Removing non-alphanumeric characters (while carefully preserving mathematical operators) and eliminating English stopwords via NLTK.

3.2 Feature Engineering Strategy

A dual-approach was adopted to represent the text data effectively.

A. Semantic Vectorization (TF-IDF):
We utilized Term Frequency-Inverse Document Frequency to transform text into a sparse matrix (limited to the top 3,000 features), capturing the weight of specific words.

B. Meta-Feature Extraction (Custom Logic):
Standard NLP vectors often miss the context of mathematical difficulty. Therefore, six custom features were engineered:

1. **Mathematical Density:** The frequency of symbols like $, \sum, ^, {, }.
2. **Keyword Detection:** Assessing the presence of terms like "Graph", "DP", "Tree".
3. **Structural Metrics:** Including Text Length, Word Count, Average Word Length, and Count of Numeric Constants.

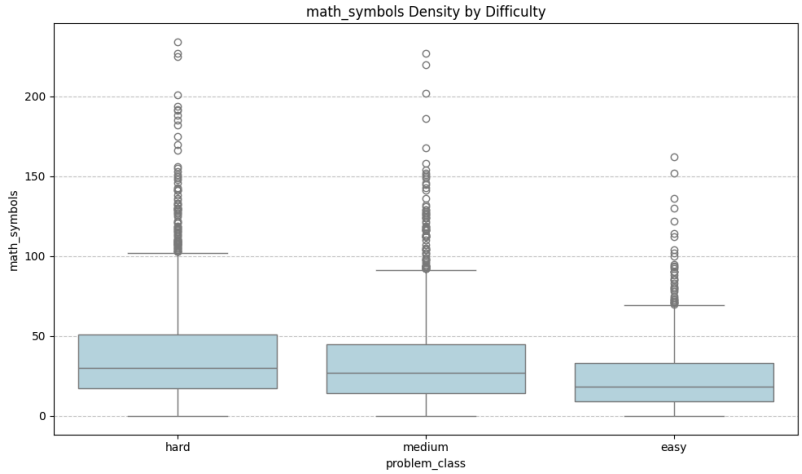

Figure 3: *Correlation Analysis (Boxplot).*
Observation: Figure 3 demonstrates a clear trend where 'Hard' problems contain a higher density of mathematical symbols, validating the feature selection.

Figure 4: *Multivariate Correlation Matrix. This heatmap illustrates the statistical dependency between the engineered structural metrics (such as word count and keyword frequency) and the final difficulty rating.*

---

4. Experimental Setup

A "Model Battle" framework was established to empirically determine the best algorithm. The data was partitioned into **80% Training** and **20% Testing** subsets.

4.1 Algorithms Evaluated

- **Classification (Categorical):** Logistic Regression, Support Vector Machine (Linear SVC), and Random Forest Classifier.
- **Regression (Numerical):** Linear Regression, Gradient Boosting Regressor, and Random Forest Regressor.

---

5. Results & Evaluation

5.1 Classification Performance

The evaluation metrics indicate that ensemble tree-based methods outperform linear separators in this domain.

Algorithm	Accuracy	Outcome
Random Forest	**55.04%**	**Selected**
Logistic Regression	51.64%	Not Selected
Linear SVC	50.55%	Not Selected

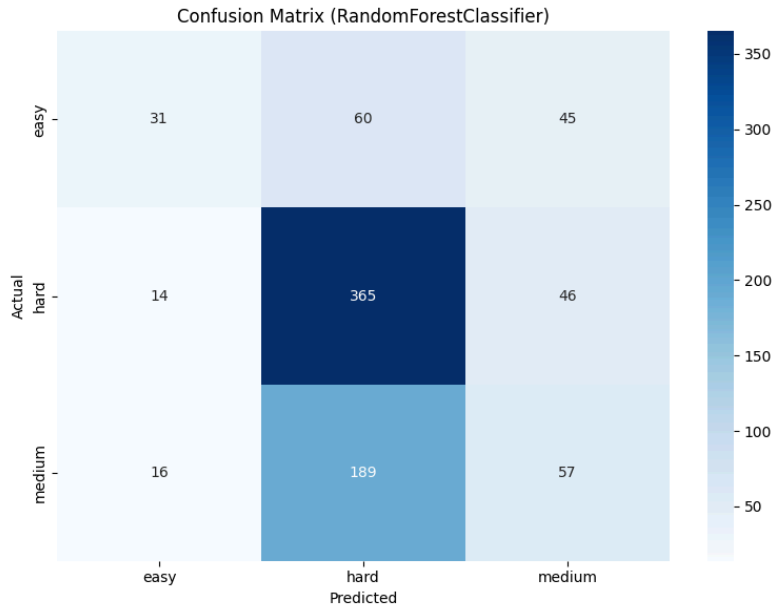

Figure 5: *Confusion Matrix for the Random Forest model.*
Interpretation: The model shows strong predictive capability along the diagonal. Some overlap exists between Medium and Hard classes, likely due to similar vocabulary usage in intermediate topics.

5.2 Regression Performance

For the score prediction task, the Random Forest model again demonstrated the highest precision (lowest error).

Algorithm	MAE (Error)	RMSE	Outcome
Random Forest	**1.68**	**2.02**	**Selected**
Gradient Boosting	1.70	2.04	Not Selected
Linear Regression	2.70	3.37	Not Selected

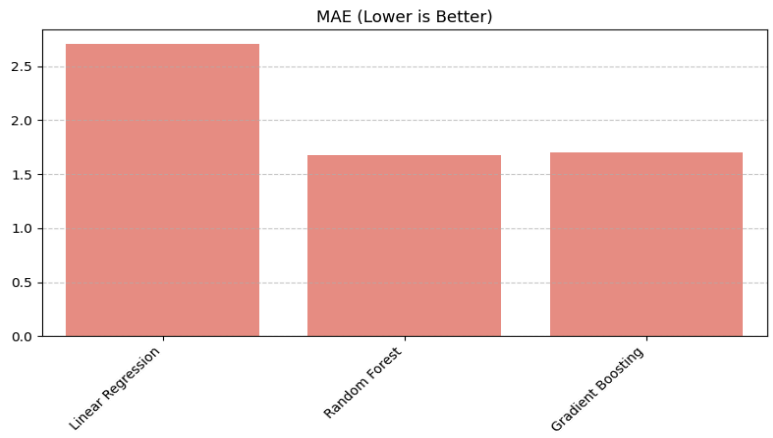

Figure 6: *Bar chart comparing Regression Error (MAE).*

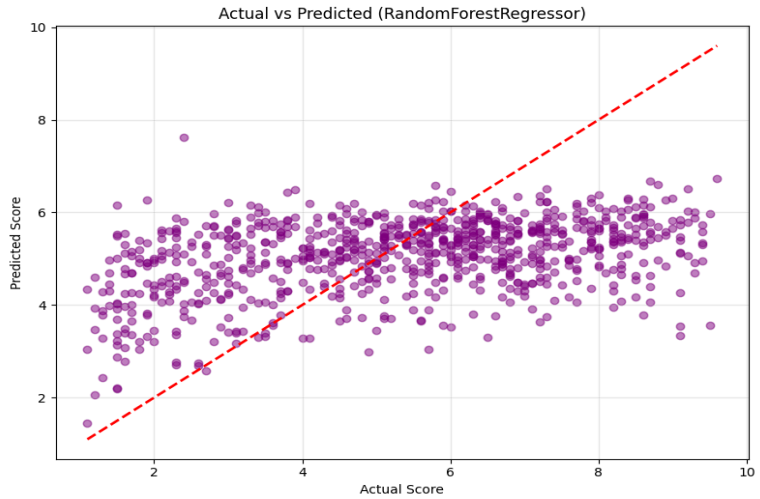

Figure 7: *Scatter plot of Predicted vs Actual Scores.*

---

6. Application Deployment

The trained models were integrated into a **Streamlit** web application to demonstrate real-time utility.

System Features:

- **Input Interface:** Allows users to paste raw problem details.
- **Live Processing:** The app replicates the backend feature extraction logic to ensure accuracy.
- **Visual Output:** Returns a color-coded difficulty class, a specific rating, and a breakdown of the detected meta-features (e.g., Math Density).

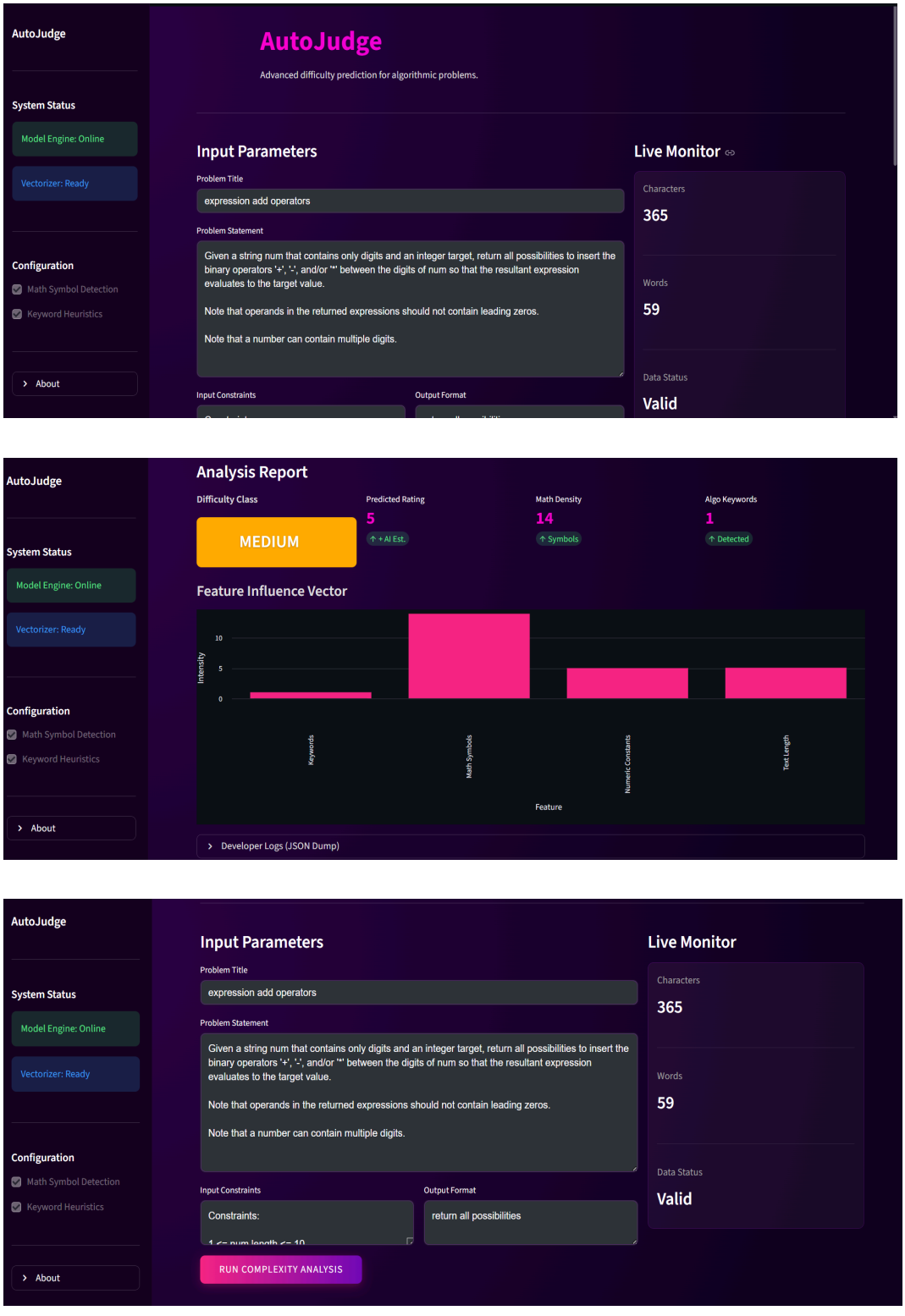

Figure 8: *Snapshots of the Analysis Dashboard.*

7. Conclusion & Future Work

The **AutoJudge** system successfully validates that combining traditional NLP with domain-specific meta-features is a viable strategy for automating problem classification.

Key Insights:

1. **Feature Impact:** Mathematical notation is a highly significant predictor of problem difficulty.
2. **Algorithm Choice:** Random Forest models handle the non-linear complexity of this dataset better than Linear Regression or SVMs.
3. **Practicality:** The resulting system provides a consistent, objective baseline for grading problems.

Future Directions:
Future iterations could incorporate Deep Learning models (such as BERT or Longformer) to better understand the semantic context of longer, narrative-based problem descriptions.

---

8. References

1. *Scikit-Learn User Guide.* Available: https://scikit-learn.org/
2. *Streamlit Documentation.* Available: https://docs.streamlit.io/
3. *NLTK Documentation.* Available: https://www.nltk.org/
4. *Pandas Data Analysis Library.* Available: https://pandas.pydata.org/